FH-OÖ Hagenberg/HSD
SDP3, WS 2026
Übung 5

Name: T. Lechner, P. Ackermann | Aufwand in h: 12

Mat.Nr: S2510306017, S2510306001 | Punkte:

Übungsgruppe: 1 | korrigiert:

Beispiel 1 (24 Punkte) Fahrsimulation: Für einen KFZ-Prüfstand ist eine Software zu entwickeln. Dabei sollen Sensorwerte zur Ermittlung der Raddrehzahlen eingelesen und verarbeitet werden. Die Aufbereitung der Daten umfasst die Berechnung der Momentangeschwindigkeit (Tachometeranzeige) und der zurückgelegten Wegstrecke (Kilometerzähler; grobe Näherung genügt).

Modellieren Sie nun einen PKW als Konkretisierung eines Kraftfahrzeuges. Der PKW hat einen Tachometer und einen Kilometerzähler. Weiters verfügt der PKW über einen Raddrehsensor, der bei jedem Abruf die momentane Raddrehzahl eines der vier Räder zurückliefert. Bei jedem Abruf der Raddrehzahl sollen die Anzeigen für den Tachometer und die Kilometeranzeige automatisch aktualisiert werden. Verwenden Sie dazu ein passendes Design Pattern.

Der Raddrehsensor wird hier in der Simulation durch eine Datei nachgebildet. Jede Zeile der Datei entspricht dabei einer momentanen Drehzahl. Kapseln Sie den Sensor in einer Klasse. In der Schnittstelle dieser Klasse befindet sich eine Methode `GetRevolutions()`, die einen Drehzahlwert von der Datei einliest und zurückgibt.

Die Klasse PKW verfügt über eine Methode `Process()`, die vom Testtreiber fix im 500ms-Takt aufgerufen wird. `Process()` liest jedes Mal einen neuen Drehzahlwert via `GetRevolutions()` ein und legt diesen Wert lokal in einem Member ab. Weiters ist eine Methode `GetCurrentSpeed()` vorzusehen, die anhand folgender technischer Daten die Momentangeschwindigkeit ermittelt:

Einheit der Raddrehzahl: U/min
Raddurchmesser: 600mm

Ermitteln Sie die Momentangeschwindigkeit in km/h nach folgender Formel:

$$\text{Geschwindigkeit (km/h)} = \frac{\text{Raddrehzahl (U/min)} \times \text{Raddurchmesser (m)} \times \pi \times 60}{1000}$$

Beispiel: 500 U/min

$$v = \frac{500}{60} \times 0{,}6 \times \pi \times 3{,}6 = 56{,}549\,\text{km/h}$$

Die Tachometer- und Kilometeranzeige holen sich im Zuge einer Benachrichtigung den aktuellen Geschwindigkeitswert. Die Kilometeranzeige berechnet auf Basis des 500ms-Taktes die gefahrene Wegstrecke:

$$s = v \times \frac{1}{7200}$$

$$s = 56{,}549 \times \frac{1}{7200} = 0{,}00785\,\text{km} = 7{,}85\,\text{m}$$

Simulieren Sie den Tachometer mit Hilfe der analogen Anzeige (AnalogDisplay) und die Kilometeranzeige mit Hilfe der digitalen Anzeige (DigitalDisplay).

Schreiben Sie einen Testtreiber, der wiederholt im 500ms-Takt die Methode `Process()` vom PKW aufruft.

Die Displays müssen nicht mitabgegeben werden. Die mitgelieferten Klassen für die Ansteuerung der Displays sind elektronisch mitabzugeben, allerdings kann auf einen Ausdruck im pdf verzichtet werden!

Treffen Sie für alle unzureichenden Angaben sinnvolle Annahmen und begründen Sie diese. Verfassen Sie weiters eine Systemdokumentation (Funktionalität, Klassendiagramm, Schnittstellen der beteiligten Klassen, etc.)!

Allgemeine Hinweise: Legen Sie bei der Erstellung Ihrer Übung großen Wert auf eine **saubere Strukturierung** und auf eine **sorgfältige Ausarbeitung!** Dokumentieren Sie alle Schnittstellen und versehen Sie Ihre Algorithmen an entscheidenden Stellen ausführlich mit Kommentaren! Testen Sie ihre Implementierungen ausführlich! Geben Sie den **Testoutput** mit ab!



Systemdokumentation Projekt Fahrsimulation

Version 1.0

T. Lechner, P. Ackermann

Hagenberg, 12. November 2026

Inhaltsverzeichnis

1 Organisatorisches **5**
1.1 Team 5
1.2 Aufteilung der Verantwortlichkeitsbereiche 5
1.3 Aufwand 6

2 Anforderungsdefinition (Systemspezifikation) **6**

3 Systementwurf **7**
3.1 Klassendiagramm 7
3.2 Designentscheidungen 7

4 Dokumentation der Komponenten (Klassen) **7**

5 Testprotokollierung **8**

6 Quellcode **8**
6.1 Object 8
6.2 ISpeedProvider 8
6.3 IWheelSpeedSensor 8
6.4 WheelSpeedSimulator 9
6.5 Vehicle 9
6.6 MotorVehicle 9
6.7 Car 10
6.8 IVehicleListener 10
6.9 Display 10
6.10 KilometerCounter 10
6.11 Speedometer 10
6.12 Testtreiber 11

1 Organisatorisches

1.1 Team

- T. Lechner, Matr.-Nr.: S2510306017, E-Mail: s2510306017@students.fh-hagenberg.at
- P. Ackermann, Matr.-Nr.: S2510306001, E-Mail: s2510306001@students.fh-hagenberg.at

1.2 Aufteilung der Verantwortlichkeitsbereiche

- T. Lechner
  - Klassendiagramm
  - Anforderungsdefinition
  - Implementierung und Unit-Test:
    - IWheelSpeedSensor
    - WheelSpeedSimulator
    - IVehicleListener
    - Display
    - Speedometer
    - KilometerCounter
- P. Ackermann
  - Designentscheidungen
  - Implementierung und Unit-Test:
    - Object

* Vehicle
* MotorVehicle
* Car
* ISpeedProvider

1.3 Aufwand

- T. Lechner: geschätzt 7 Ph / tatsächlich 0 Ph
- P. Ackermann: geschätzt 7 Ph / tatsächlich 0 Ph

2 Anforderungsdefinition (Systemspezifikation)

Das Ziel ist es, eine Software zu entwickeln, mit der Daten von Fahrzeugen (in diesem Fall einem PKW) auf einem Fahrzeug-Prüfstand gespeichert werden können. Dabei wird über einen Raddrehsensor die Drehzahl von einem der Räder gemessen und basierend darauf die Momentangeschwindigkeit und die insgesamt am Prüfstand gefahrenen Kilometer ausgegeben.
Die Ausgabe und Visualisierung der Daten soll von zwei fertigen Display-Programmen durchgeführt werden. Alle 500ms soll der Sensor überprüft, und die Daten aktualisiert werden. Ein PKW hat einen fixen Raddurchmesser von 600mm.

3 Systementwurf

3.1 Klassendiagramm

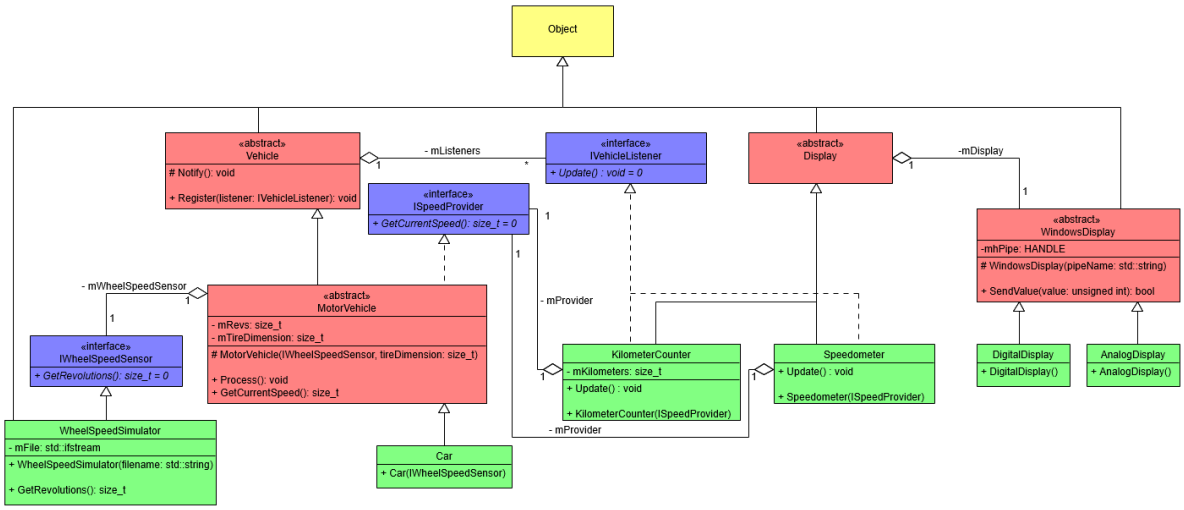


Abbildung 1: Klassendiagramm Fahrsimulation

3.2 Designentscheidungen

4 Dokumentation der Komponenten (Klassen)

Die HTML-Startdatei befindet sich im Verzeichnis ./../doxy/html/index.html

5 Testprotokollierung

6 Quellcode

6.1 Object

Listing 1: "Beispiel01/Object.h"

```
1  #ifndef OBJECT_H
2  #define OBJECT_H
3
4  /// @author Philip
5  /// @date 12.11.2026
6  /// @brief Base class of all base classes.
7  ///
8  /// Is the common root class of all classes. If there is no other
       base class,
9  /// this class is to be selected as the base class.
10 class Object
11 {
12 public:
13     /// @brief Virtual default destructor -> once virtual, always
           virtual.
14     ///
15     /// It is not necessary to declare another virtual destructor
16     /// in derived classes.
17     virtual ~Object() = default;
18
19 protected:
20     /// @brief Protected default constructor that makes the class
           abstract.
21     Object() = default;
22 };
23 #endif
```

6.2 ISpeedProvider

Listing 2: "Beispiel01/ISpeedProvider.h"

```
1  #pragma once
```

6.3 IWheelSpeedSensor

Listing 3: "Beispiel01/IWheelSpeedSensor.h"

6.4 WheelSpeedSimulator

Listing 4: Beispiel01/WheelSpeedSimulator.h

```
1 #pragma once
```

Listing 5: Beispiel01/WheelSpeedSimulator.cpp

6.5 Vehicle

Listing 6: Beispiel01/Vehicle.h

```
1 #pragma once
```

Listing 7: Beispiel01/Vehicle.cpp

6.6 MotorVehicle

Listing 8: Beispiel01/MotorVehicle.h

```
1 #pragma once
```

Listing 9: Beispiel01/MotorVehicle.cpp

6.7 Car

Listing 10: Beispiel01/Car.h

```
1 #pragma once
```

Listing 11: Beispiel01/Car.cpp

```
```

6.8 IVehicleListener

Listing 12: "Beispiel01/IVehicleListener.h"

```
1 #pragma once
```

6.9 Display

Listing 13: "Beispiel01/Display.h"

```
1 #pragma once
```

6.10 KilometerCounter

Listing 14: Beispiel01/KilometerCounter.h

```
1 #pragma once
```

Listing 15: Beispiel01/KilometerCounter.cpp

```
```

6.11 Speedometer

Listing 16: Beispiel01/Speedometer.h

```
1 #pragma once
```

Listing 17: Beispiel01/Speedometer.cpp

6.12 Testtreiber

Listing 18: "Beispiel01/Test.cpp"

```
1 #include "gtest/gtest.h"
2
3 TEST(TestCaseName, TestName) {
4   EXPECT_EQ(1, 1);
5   EXPECT_TRUE(true);
6 }
```